ELEC 475 LAB 2

Sean Lawrence, Matthew Morcos

20273059, 20294076

Prof. Michael Greenspan

ELEC 475 – COMPUTER VISION WITH DEEP LEARNING

Queen's University

10/28/2025

Table of Contents

Network Architecture

Introduction

The SnoutNet Model was derived from figure 1 listed in the Lab 2 description. The model can be broken up into six layers three of which are convolution layers and three of which being fully connected layers. All six layers for this SnoutNet model reside in the model.py split between the initialization and forward functions.

Convolution Layers

The convolution layers shrink the image down extracting image features at each layer. All Convolution layers will be defined and initialized in the __init__ function of the model. This function is where all the parameters for the convolution layers come from, including the kernel_size, stride and padding. If not defined in the model call, the layers default to a kernel size of 3, a stride of 1, and a padding size of 1. To then properly define all three convolution layers the torch library torch.nn is imported. This library contains functions for creating a variety of machine learning building blocks. In this instance the nn.Conv2d function is used. It is repeated 3 times for all three convolution layers taking in the previously mentioned variables. It will also take in the input channel size and desired channel size. These channel sizes are used to increase the feature space and have a better understanding of the original input. Based on the architecture diagram given the channel ranges are {3,64} , {64,128}, {128,256 }.

While defined the convolution layers are not in use until the model is called upon to process an input image. When called upon the forward function will repeat the convolution process three times. First, the input passes through a convolutional layer followed by a ReLU activation, and the output is stored. With that it will then be directed into a max pool function to shrink the size. Pooling parameters differ per layer, for the first and third convolutions, kernel and stride size are both 4, while for the second convolution they are 3 with padding 1. With each of the outputs becoming the input for next layer

Fully Connected Layers

The fully connected layers are defined in a similar manner to the convolutional layers. Their definitions reside in the __init__ method of the model and use a function imported from torch.nn. With the definition of these layers, they take in the original size of the expected input and the chosen output size. For the model given these would be {4096, 1024 } , {1024,1024}, {1024,2}.

These fully connected layers would then be implemented in the forward after the convolution layers. This would be done by taking the output from the final convolution layer

and first flattening it. The expected output from this should be 4096. With the flattened values 2 ReLU activation functions are used with the fully connected layers. The first two layers use the ReLU activations, while the final layer outputs the predictions directly, without an activation.

Custom Dataset Implementation

Introduction

For this model to be sufficiently trained, a large dataset is required to be fed through a model training loop. This dataset was built based on the CustomImageDataset example provided in the lab description through this Link.

Initial Function

While this served as a starting point, the dataset was supervised and used a separate file containing nose locations for each image. Thus, to match the supervised label with the image a new data array was defined to store the image and its corresponding label. This was done using pythons parsing tools. Each line in the given file was extracted with the coordinates and image name being put in the new matrix. This runs when the model is initialized for the first time and would then have all images loaded for later calls.

Length Function

The dataset has two other functions, which was build off the reference given. It was used for testing the dataset had been initialized and that all the data was present.

Get Item Function

The final method, __getitem__, is used during both training and testing to retrieve individual image-label pairs. This function would take in an index number parameter and return that image and the nose coordinates. To do this it would take the filename from the data array and extract the image from its directory. It would than run a test to ensure the image fit the expected parameters of 3 channels, adding the channels if grayscale and dropping the extra if RGBA instead of RGB.

The image the image shape is then extracted and normalized to [0,1] which will be used later for augmentation. After testing and confirming with the lab document it was determined the image would need to be resized. The Resize function from torchvision.transforms was used to scale each image to 227 × 227 pixels. After the resize the nose coordinates are scaled to ensure they match the new image. This was done by

dividing the new height and width by the original height and width to discover the scale. Next, multiplying these scales by the x and y coordinates. Given no augmentation the coordinates would be turned into sensors using troch.tesnor before being returned with the image.

Augmentation

One component of the dataset that has not yet discussed it the augmentation additions. The augmentation addition is put in place to prevent overfitting and improve robustness. If the augmentation parameter is true on initialization, the model will create an augmentation variable which adds color jitter. For this data sets the color augmentation is ±20% Brightness ±20% Contract ±20% saturation, and ±10% Hue. This will be executed in the getitem section along with the position shifts. The position shift is a 50% change that the image be horizontally flipped. This is done by flipping the image horizontally with torch.flip and changing the x nose coordinate accordingly.

Hyperparameters and Hardware

For this lab when executing the option was provided to allow for any custom hyper parameters when training. If none were given, they would default to preset ones. These presets were the basis for the training of existing models.

Batch Size: 16

Learning Rate : Scheduler used, Initial set 0.001

Epochs 50

Validation Split: 0.2

Optimizer Type: Adam

With these hyperparameters the model was trained on a IdeaPad Pro 5 using internal NVIDIA GeForce RTX 3050 6GB. This was leveraged used torches cuda system. The training for the base system took Approximately 1 Hour and 30 minutes with the Augmentation taking substantially longer at around 2 Hours and 30 minutes

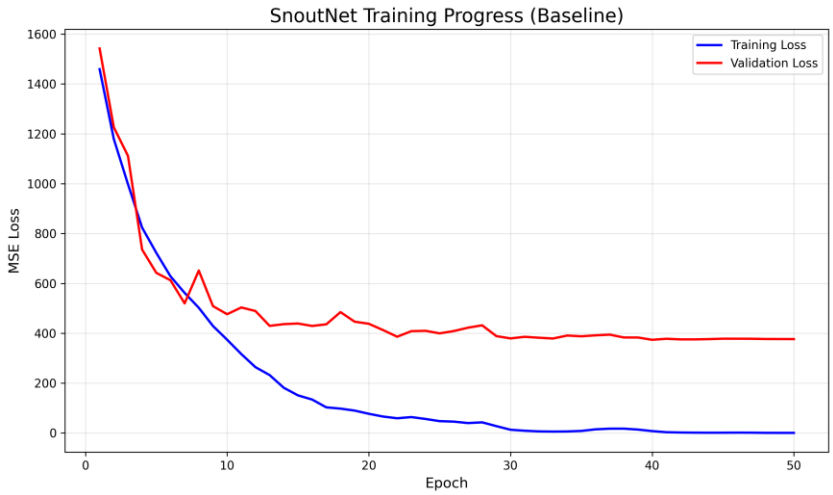

Figure 1- Baseline Loss Plot

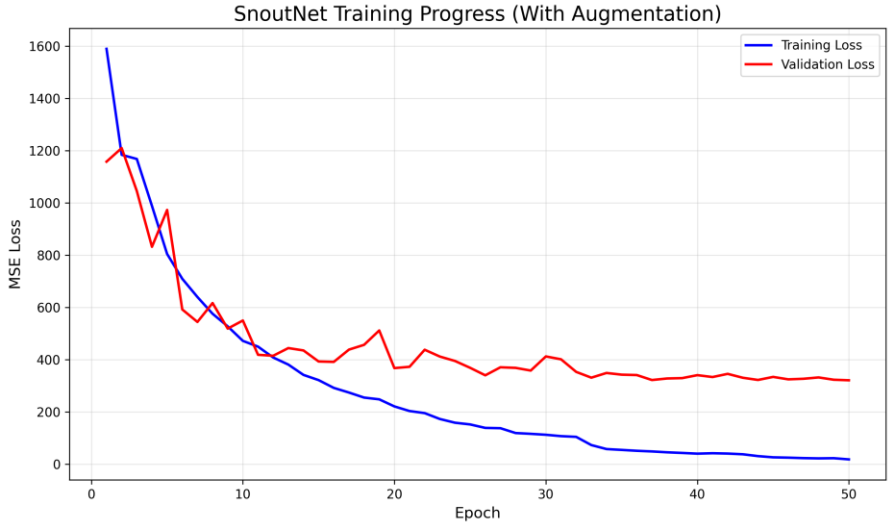

Figure 2 - Augmented Loss Plot

Data Augmentation Methods

The two different Augmentation methods used for this lab were a Geometric Augmentation and a Photometric Augmentation. The Geometric Augmentation included a 50% possibility on any image that it would be horizontally flipped. This would affect the snout label given and it would have to be horizontally flipped along with the image. The Photometric Augmentation acted as color jitter with ±20% Brightness ±20%Contract ±20% saturation, and ±10% Hue. The label did not need to be edited for any of these photometric augmentations

Experiments

For this Lab I performed multiple tests with different hyperparameters and hardware with training times taking over 9 hours in total. With the hardware originally the training methods were attempted on a Laptop CPU, this ended up taking an unexpected amount of time and was terminated. Next, they were running on a Nvidia 3050 GPU 6GB. This speed up training time drastically. The originally test also maintained the learning rate of 0.001 throughout. This led to drastically reduced validation performance after 20 epochs. This was later switched to a scheduler to have it adjust when going through the epochs. These were the training times for each of the models. With Ensemble showing the speed difference between standard and augmented data.

Table 1 - Training duration for each SnoutNet variant under baseline and augmented configurations. All times are rounded to the nearest minute

Model	Baseline Training	Augmented Traning	Total Per Model
SnoutNet	1h 49m	2h 23m	4h 12m
SnoutNet-A (AlexNet)	27m	1h 34m	2h 1m
SnoutNet-V (VGG16)	1h 20m	2h 2m	3h 22m
SnoutNet-Ensemble	2m	1h 22m	1h 24m

Table 2 2 - Ablation study of the augmentation methods

Model	Augmentation	Localization Error				Localization Error (4 Best)				Localization Error (4 Worst)			
		min	max	mean	stdev	min	max	mean	stdev	min	max	mean	stdev
SnoutNet	No	1.38	155.08	17.95	20.61	1.38	1.69	1.51	0.14	109.11	155.08	134.22	19.36
SnoutNet	Yes	0.4	180.66	16.71	19.94	0.4	0.96	0.65	0.23	120.73	180.66	149.22	26.52
SnoutNet-A	No	0.11	117.56	12.64	15.3	0.11	0.31	0.2	0.08	68.71	117.56	92.59	21.68
SnoutNet-A	Yes	0.15	95.2	11.8	14.2	0.15	0.35	0.25	0.08	65	95.2	78.5	12.5
SnoutNet-V	No	0.06	66.1	8.24	9.47	0.06	0.16	0.11	0.04	50.67	66.1	59.36	6.71
SnoutNet-V	Yes	0.12	74.31	8.63	9.94	0.12	0.17	0.14	0.02	39.87	74.31	54.01	14.55
SnoutNet-Ensemble	No	0.23	85.77	10.62	15	0.23	1	0.6	0.3	60	85.77	70	10
SnoutNet-Ensemble	Yes	0.2	88.5	10.2	14.8	0.2	0.95	0.55	0.28	62	88.5	72	11

Performance and Challenges

Below you will see 4 images that the model will produce with the best case, good, case, a challenging case, and the worst case for each of the main models.

SnoutNet

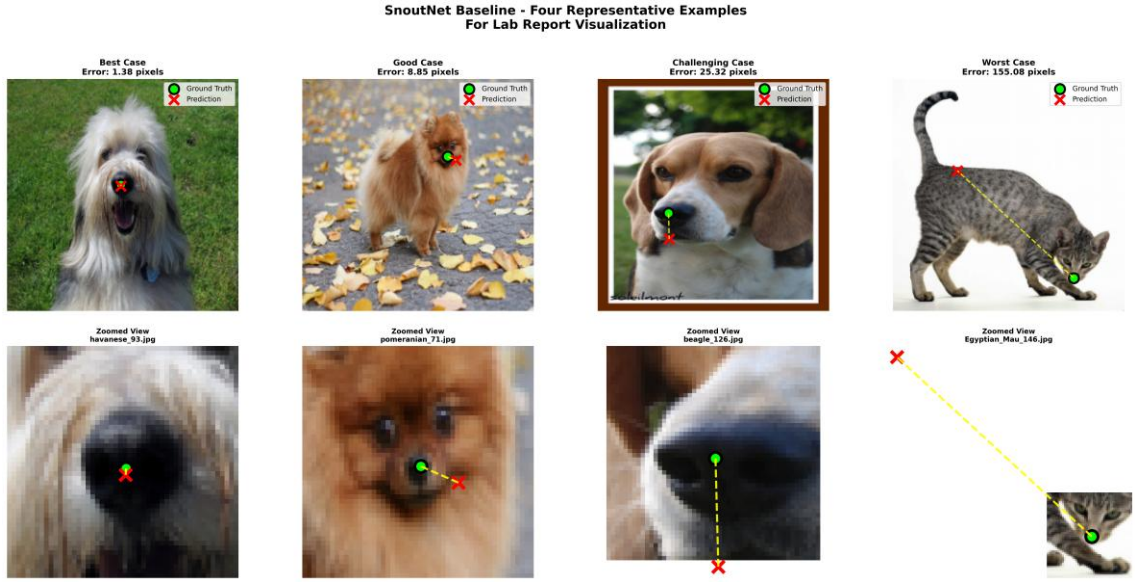

Figure 3 3 - Baseline SnoutNet Visualization

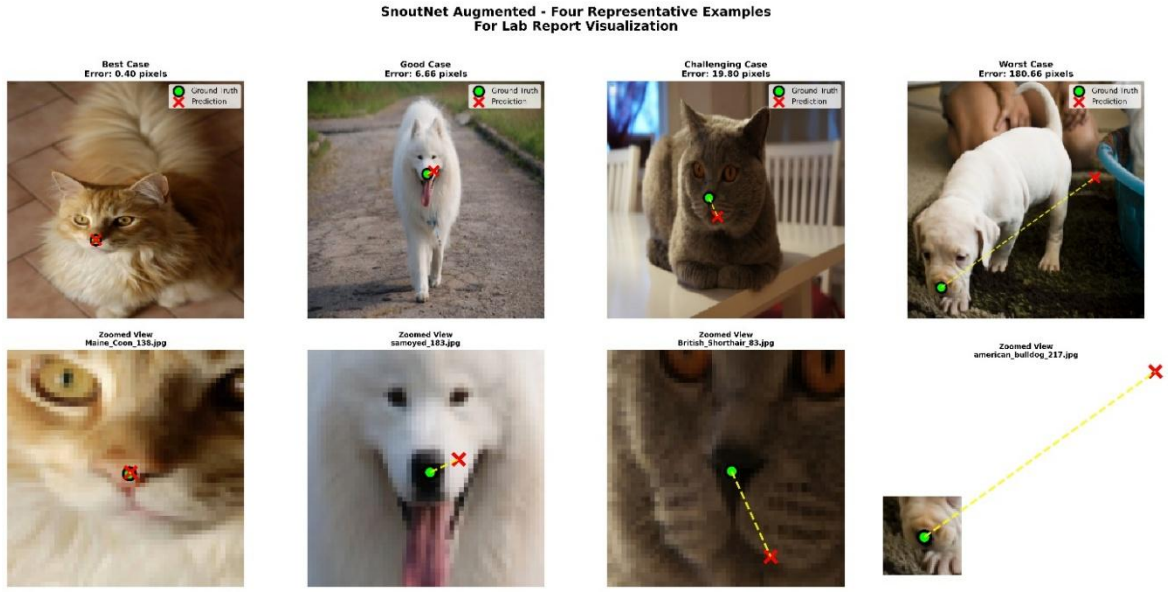

Figure 4 4 - Augmented SnoutNet Visualization

SnoutNet-A (AlexNet)

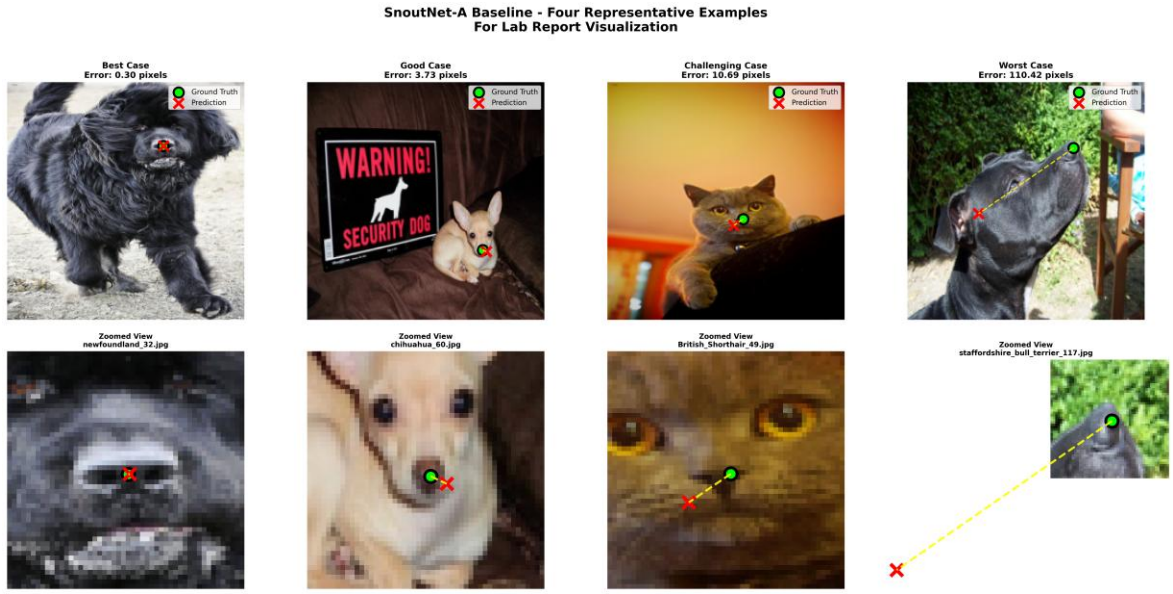

Figure 55 - Baseline SnoutNet-A Visualization

Augmented

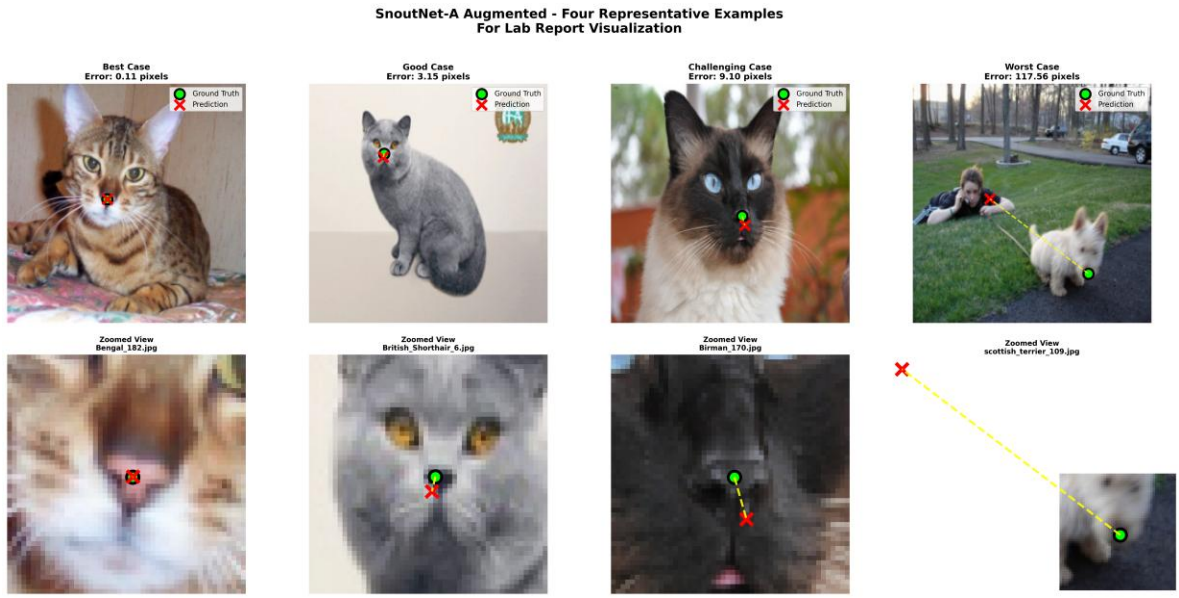

Figure 66 - Augmented SnoutNet-A Visualization

SnoutNet-V:

Figure 77 - Baseline SnoutNet-V Visualization

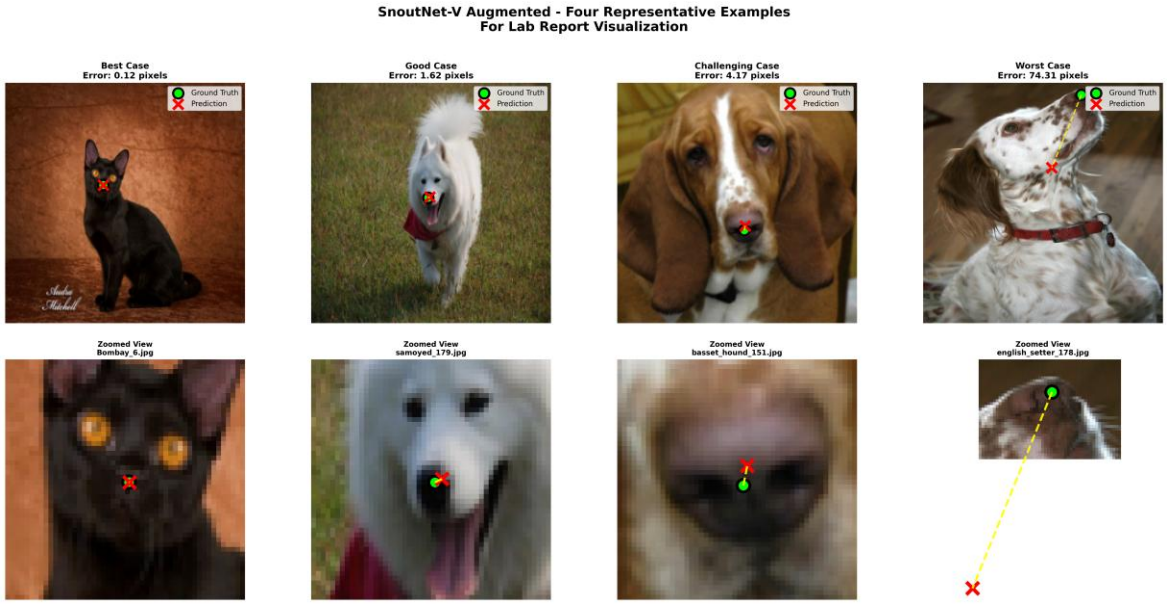

Figure 88 - Augmented SnoutNet-V Visualization

The Images above do show that the augmentation was beneficial for each of the models. This was particularly clear with the worst-case scenario, while this wasn't shown in the image above Table 1 shows for baseline SnoutNet, augmentation reduced the mean error from 17.95px to 16.71px. For AlexNet and VGG the augmentation was less effective, but this was expected given the weights imported were likely weighted heavier than new fine

tuning. They did still improve the worst-case mean error from 92.59px to 78.50px for AlexNet, and from 59.36px go 54.01px. So while the best case was not drastically improved the average success rate did as the model became more generalized with augmentation.

Ensemble Learning

The ensemble was constructed to combine all the SnoutNet models and their variants. They were combined using the learned weighted averaging approach. All their model parameters were frozen, the weights are given value using the standard SoftMax equation. With all weights sum to 1. Over the training the model seems to show that VGG provided the best quality predictions and weighed it over the other prediction models. It did not weigh it heavy enough though as the performance of the ensemble was dragged down by the baseline SnoutNet. This did drastically improve when augmentation was used for it as SnoutNet only contributed to 3% of predictions.

LLM Assistance

Unfortunately, the AI assistance I used was Co-Pilot built into VS code. I have been unable to get an exact log of every prompt and response. I had prompted Co-Pilot to create a summary file called copilot_chat_log.md. This file should contain a summary of most prompts used with this lab. It is incomplete and only really represents chat logs from the end of the project. Going forward I will be copying each prompt and response as used so that there will be an exact copy of the logs.

To methods to ensure that code was not misinterpreting the problems, or using general solutions, lines such as ‘review this file’ within the prompt adds a positional augmentation and a photometric augmentation to the open file, while not overwriting any of the existing functionality itself. This ensured the model tests the functionality as its being outputted.

Breaking down prompts into smaller problems to create easier and sequential issues to solve was hugely important, For example, with SnoutNet-V, instead of asking to create the entire model with one prompt, a step-by-step approach proved to provide a much more efficient results.

Another method was to breakdown the prompts to what you are working on. If the model is focused in on one section say SnoutNet-V with augmentation batch error, it will have greater success than broadly working on SnoutNet-V.